02. dstack Guest Agent - 초기화 과정 상세 분석

목차

1. Guest Agent 아키텍처 개요
2. 메인 함수 실행 과정
3. AppState 초기화
4. 서비스 구성 요소
5. 네트워크 바인딩
6. Watchdog 시스템
7. 설정 관리
8. 인증서 관리

---

1. Guest Agent 아키텍처 개요

1.1 핵심 구조체

```
// guest-agent/src/rpc_service.rs
#[derive(Clone)]
pub  struct AppState {
inner: Arc<AppStateInner>,
}
```

```
struct AppStateInner {
config: Config, // 설정 정보
keys: AppKeys, // 애플리케이션 키
vm_config: String, // VM 설정 정보
cert_client: CertRequestClient, // 인증서 요청 클라이언트
demo_cert: String, // 데모 인증서
}
```

1.2 서비스 구성

Guest Agent는 4개의 독립적인 서비스로 구성됩니다:

1. **Internal V0 Service**: 레거시 호환성을 위한 내부 RPC 서비스
2. **Internal Service**: 최신 내부 RPC 서비스
3. **External Service**: 외부 접근용 HTTP/RPC 서비스
4. **Guest API Service**: VM 간 통신용 API 서비스

---

2. 메인 함수 실행 과정

2.1 초기화 시퀀스

```
// guest-agent/src/main.rs
#[rocket::main]
```

```rust
async fn main() -> Result<()> {
// 1. 로깅 시스템 초기화
{
use tracing_subscriber::{fmt, EnvFilter};
let filter = EnvFilter::try_from_default_env()
.unwrap_or_else(|_| EnvFilter::new("info"));
fmt().with_env_filter(filter).init();
}
// 2. 명령행 인수 파싱
let args = Args::parse();
// 3. 설정 파일 로딩
let figment = config::load_config_figment(args.config.as_deref());
// 4. AppState 초기화
let state = AppState::new(figment.focus("core").extract()?)
.await
.context("Failed to create app state")?;
// 5. 각 서비스별 설정 추출
let internal_v0_figment = figment.clone().select("internal-v0");
let internal_figment = figment.clone().select("internal");
let external_figment = figment.clone().select("external");
let guest_api_figment = figment.select("guest-api");
// 6. 외부 바인딩 주소 추출
let bind_addr: BindAddr = external_figment
```

```
.extract()

.context("Failed to extract bind address")?;

// 7. 서비스 준비 완료 신호용 채널 생성

let (tappd_ready_tx, tappd_ready_rx) = oneshot::channel();

let (sock_ready_tx, sock_ready_rx) = oneshot::channel();

// 8. 모든 서비스 동시 실행

tokio::select!(

res = run_internal_v0(state.clone(), internal_v0_figment, tappd_ready_tx) => res

res = run_internal(state.clone(), internal_figment, sock_ready_tx) => res?,

res = run_external(state.clone(), external_figment) => res?,

res = run_guest_api(state.clone(), guest_api_figment) => res?,

_ = async {

let  _ = tappd_ready_rx.await;

let  _ = sock_ready_rx.await;

if  args.watchdog {

run_watchdog(bind_addr.port).await;

} else {

pending::<()>().await;

}

} => {}

);

Ok(())
```

```
}
```

2.2 명령행 인수 구조

```
// guest-agent/src/main.rs
#[derive(Parser)]
#[command(author, version, about, long_version = app_version())]
struct Args {
/// Path to the configuration file
#[arg(short, long)]
config: Option<String>,

/// Enable systemd watchdog
#[arg(short, long)]
watchdog: bool,
}

fn  app_version() -> String {
format!("v{CARGO_PKG_VERSION} ({GIT_REV})")
}

const CARGO_PKG_VERSION: &str = env!("CARGO_PKG_VERSION");
const GIT_REV: &str = git_version::git_version!(
```

```
args = ["--abbrev=20", "--always", "--dirty=-modified"],

prefix = "git:",

fallback = "unknown"

);
```

---

3. AppState 초기화

3.1 초기화 과정

```
// guest-agent/src/rpc_service.rs

impl AppState {

pub  async  fn  new(config: Config) -> Result<Self> {

// 1. 애플리케이션 키 로딩

let  keys: AppKeys = serde_json::from_str(&fs::read_to_string(&config.keys_file)

.context("Failed to parse app keys")?;

// 2. 시스템 설정 로딩

let  sys_config: SysConfig =

serde_json::from_str(&fs::read_to_string(&config.sys_config_file)?)

.context("Failed to parse VM config")?;

let  vm_config = sys_config.vm_config;

// 3. 인증서 요청 클라이언트 생성

let  cert_client =

CertRequestClient::create(&keys, config.pccs_url.as_deref(), vm_config.clone())
```

```
.await
.context("Failed to create cert signer")?;
// 4. 데모 키 생성
let  key = KeyPair::generate().context("Failed to generate demo key")?;
// 5. 데모 인증서 요청
let  demo_cert = cert_client
.request_cert(
&key,
CertConfig {
org_name: None,
subject: "demo-cert".to_string(),
subject_alt_names: vec![],
usage_server_auth: false,
usage_client_auth: true,
ext_quote: true,
},
config.simulator.enabled,
)
.await
.context("Failed to get app cert")?
.join("\n");
// 6. AppState 구성
Ok(Self {
```

```
inner: Arc::new(AppStateInner {
config,
keys,
cert_client,
demo_cert,
vm_config,
}),
})
}

pub fn config(&self) -> &Config {
&self.inner.config
}
}
```

3.2 키 파일 구조

```
// dstack_types/src/lib.rs (추정)
#[derive(Serialize, Deserialize)]
pub struct AppKeys {
pub k256_key: Vec<u8>, // K256 개인키
pub k256_signature: Vec<u8>, // K256 서명
// 기타 키 정보들
```

```
}
```

---

4. 서비스 구성 요소

4.1 Internal V0 Service (레거시)

```
// guest-agent/src/main.rs

async fn run_internal_v0(

state: AppState,

figment: Figment,

sock_ready_tx: oneshot::Sender<()>,

) -> Result<()> {

// 1. Rocket 웹 프레임워크 설정

let rocket = rocket::custom(figment)

.mount(

"/prpc/",

ra_rpc::prpc_routes!(AppState, InternalRpcHandlerV0, trim: "Tappd."),

)

.manage(state);

// 2. Rocket 점화

let ignite = rocket

.ignite()
```

```
.await
.map_err(|err| anyhow!("Failed to ignite rocket: {err}"))?;
// 3. 바인딩 엔드포인트 확인
let  endpoint = DefaultListener::bind_endpoint(&ignite)
.map_err(|err| anyhow!("Failed to get endpoint: {err}"))?;
// 4. 리스너 바인딩
let  listener = DefaultListener::bind(&ignite)
.await
.map_err(|err| anyhow!("Failed to bind on {endpoint}: {err}"))?;
// 5. Unix 소켓 권한 설정
if  let Some(path) = endpoint.unix() {
// 모든 사용자가 소켓에 연결할 수 있도록 권한 설정
fs_err::set_permissions(path, Permissions::from_mode(0o777))?;
}
// 6. 준비 완료 신호 전송
sock_ready_tx.send(()).ok();
// 7. 서비스 시작
ignite
.launch_on(listener)
.await
.map_err(|err| anyhow!(err.to_string()))?;
Ok(())
```

```
}
```

4.2 Internal Service (최신)

```
// guest-agent/src/main.rs
async fn run_internal(
state: AppState,
figment: Figment,
sock_ready_tx: oneshot::Sender<()>,
) -> Result<()> {
let rocket = rocket::custom(figment)
.mount("/", ra_rpc::prpc_routes!(AppState, InternalRpcHandler))
.manage(state);
let ignite = rocket
.ignite()
.await
.map_err(|err| anyhow!("Failed to ignite rocket: {err}"))?;
let endpoint = DefaultListener::bind_endpoint(&ignite)
.map_err(|err| anyhow!("Failed to get endpoint: {err}"))?;
let listener = DefaultListener::bind(&ignite)
.await
.map_err(|err| anyhow!("Failed to bind on {endpoint}: {err}"))?;
if let Some(path) = endpoint.unix() {
```

```
fs_err::set_permissions(path, Permissions::from_mode(0o777))?;
}
sock_ready_tx.send(()).ok();
ignite
.launch_on(listener)
.await
.map_err(|err| anyhow!(err.to_string()))?;
Ok(())
}
```

4.3 External Service

```
// guest-agent/src/main.rs
async fn run_external(state: AppState, figment: Figment) -> Result<()> {
let rocket = rocket::custom(figment)
// HTTP 라우트 마운트
.mount("/", http_routes::external_routes(state.config()))
// RPC 라우트 마운트
.mount(
"/prpc",
ra_rpc::prpc_routes!(AppState, ExternalRpcHandler, trim: "Worker."),
)
// 응답 헤더에 앱 버전 추가
```

```
.attach(AdHoc::on_response("Add app version header", |_req, res| {
Box::pin(async move {
res.set_raw_header("X-App-Version", app_version());
})
}))
.manage(state);
let _ = rocket
.launch()
.await
.map_err(|err| anyhow!("Failed to ignite rocket: {err}"))?;
Ok(())
}
```

4.4 Guest API Service

```
// guest-agent/src/main.rs
async fn run_guest_api(state: AppState, figment: Figment) -> Result<()> {
let rocket = rocket::custom(figment)
.mount("/api", ra_rpc::prpc_routes!(AppState, GuestApiHandler))
.manage(state);

let ignite = rocket
.ignite()
```

```
.await
.map_err(|err| anyhow!("Failed to ignite rocket: {err}"))?;
// 기본 리스너 또는 VsockListener 사용
if DefaultListener::bind_endpoint(&ignite).is_ok() {
let  listener = DefaultListener::bind(&ignite)
.await
.map_err(|err| anyhow!("Failed to bind guest API : {err}"))?;
ignite
.launch_on(listener)
.await
.map_err(|err| anyhow!(err.to_string()))?;
} else {
let  listener = VsockListener::bind_rocket(&ignite)
.map_err(|err| anyhow!("Failed to bind guest API : {err}"))?;
ignite
.launch_on(listener)
.await
.map_err(|err| anyhow!(err.to_string()))?;
}
Ok(())
}
```

5. 네트워크 바인딩

5.1 바인딩 주소 설정

```
// guest-agent/src/config.rs (추정)
#[derive(Deserialize)]
pub struct BindAddr {
pub port: u16,
pub address: Option<String>,
}
```

5.2 Unix 소켓 권한 설정

```
// guest-agent/src/main.rs
if let Some(path) = endpoint.unix() {
// 모든 사용자가 소켓에 연결할 수 있도록 권한 설정 (0o777)
fs_err::set_permissions(path, Permissions::from_mode(0o777))?;
}
```

5.3 VsockListener 사용

```
// VsockListener는 VM 간 통신을 위한 특별한 리스너
let listener = VsockListener::bind_rocket(&ignite)
```

```
.map_err(|err| anyhow!("Failed to bind guest API : {err}"))?;
```

---

6. Watchdog 시스템

6.1 Systemd Watchdog 통합

```
// guest-agent/src/main.rs

async fn run_watchdog(port: u16) {

let mut watchdog_usec = 0;

// 1. Systemd에서 watchdog 활성화 여부 확인

let enabled = sd_notify::watchdog_enabled(false, &mut watchdog_usec);

if !enabled {

info!("Watchdog is not enabled in systemd service");

return pending::<()>().await;

}


info!("Starting watchdog");

// 2. Systemd에 준비 완료 알림

if let Err(err) = sd_notify(false, &[NotifyState::Ready]) {

error!("Failed to notify systemd: {err}");

}

// 3. 하트비트 간격 계산 (watchdog 타임아웃의 절반)

let heatbeat_interval = Duration::from_micros(watchdog_usec / 2);
```

```
let heatbeat_interval = heatbeat_interval.max(Duration::from_secs(1));
info!("Watchdog enabled, interval={watchdog_usec}us, heartbeat={heatbeat_interva
let mut interval = tokio::time::interval(heatbeat_interval);
let probe_url = format!("http://localhost:{port}/prpc/Worker.Version");
// 4. 주기적 헬스 체크 및 하트비트
loop {
interval.tick().await;


let client = reqwest::Client::new();
match client.get(&probe_url).send().await {
Ok(response) if response.status().is_success() => {
// 헬스 체크 성공 시 Systemd에 하트비트 전송
if let Err(err) = sd_notify(false, &[NotifyState::Watchdog]) {
error!("Failed to notify systemd: {err}");
}
}
Ok(response) => {
error!("Health check failed with status: {}", response.status());
}
Err(err) => {
error!("Health check request failed: {err:?}");
}
}
```

```
}

}
```

6.2 헬스 체크 엔드포인트

Watchdog는 `Worker.Version` RPC 엔드포인트를 사용하여 서비스 상태를 확인합니다:

```
// guest-agent/src/rpc_service.rs의 ExternalRpcHandler

impl WorkerRpc for ExternalRpcHandler {

async fn version(self) -> Result<WorkerVersion> {

Ok(WorkerVersion {

version: env!("CARGO_PKG_VERSION").to_string(),

rev: super::GIT_REV.to_string(),

})

}

}
```

---

7. 설정 관리

7.1 설정 파일 로딩

```
// guest-agent/src/config.rs (추정)

pub fn load_config_figment(config_path: Option<&str>) -> Figment {
```

```rust
let mut figment = Figment::new();
// 기본 설정 로딩
if let Some(path) = config_path {
figment = figment.merge(Toml::file(path));
}
// 환경 변수 오버라이드
figment = figment.merge(Env::prefixed("DSTACK_"));
figment
}
```

7.2 설정 구조체

```rust
// guest-agent/src/config.rs (추정)
#[derive(Deserialize)]
pub struct Config {
pub keys_file: String, // 키 파일 경로
pub sys_config_file: String, // 시스템 설정 파일 경로
pub pccs_url: Option<String>, // PCCS 서버 URL
pub simulator: SimulatorConfig, // 시뮬레이터 설정
}

#[derive(Deserialize)]
pub struct SimulatorConfig {
```

```
pub enabled: bool, // 시뮬레이터 모드 활성화
pub quote_file: String, // Quote 파일 경로
pub event_log_file: String, // 이벤트 로그 파일 경로
}
```

---

8. 인증서 관리

8.1 CertRequestClient 초기화

```
// cert-client 라이브러리 사용 (추정)
let cert_client = CertRequestClient::create(
&keys, // 애플리케이션 키
config.pccs_url.as_deref(), // PCCS URL
vm_config.clone() // VM 설정
)
.await
.context("Failed to create cert signer")?;
```

8.2 데모 인증서 생성

```
// guest-agent/src/rpc_service.rs
let key = KeyPair::generate().context("Failed to generate demo key")?;
let demo_cert = cert_client
```

```
.request_cert(
&key,
CertConfig {
org_name: None,
subject: "demo-cert".to_string(),
subject_alt_names: vec![],
usage_server_auth: false,
usage_client_auth: true,
ext_quote: true, // TDX Quote 확장 포함
},
config.simulator.enabled,
)
.await
.context("Failed to get app cert")?
.join("\n");
```

8.3 TLS 키 생성 서비스

```
// guest-agent/src/rpc_service.rs
impl DstackGuestRpc for InternalRpcHandler {
async fn get_tls_key(self, request: GetTlsKeyArgs) -> anyhow::Result<GetTlsKey
// 1. 보안 시드 생성
let mut seed = [0u8; 32];
```

```
SystemRandom::new()
.fill(&mut  seed)
.context("Failed to generate secure seed")?;
// 2. ECDSA 키 쌍 유도
let  derived_key =
derive_ecdsa_key_pair_from_bytes(&seed, &[]).context("Failed to derive key")?;
// 3. 인증서 설정
let  config = CertConfig {
org_name: None,
subject: request.subject,
subject_alt_names: request.alt_names,
usage_server_auth: request.usage_server_auth,
usage_client_auth: request.usage_client_auth,
ext_quote: request.usage_ra_tls,
};
// 4. 인증서 요청
let  certificate_chain = self
.state
.inner
.cert_client
.request_cert(&derived_key, config, self.state.config().simulator.enabled)
.await
.context("Failed to sign the CSR")?;
```

```
Ok(GetTlsKeyResponse {
key: derived_key.serialize_pem(),
certificate_chain,
})
}
}
```

요약

dstack Guest Agent의 초기화 과정은 다음과 같은 특징을 가집니다:

핵심 특징:

1. **다중 서비스 아키텍처**: 4개의 독립적인 서비스가 동시에 실행
2. **비동기 초기화**: tokio::select!를 사용한 병렬 서비스 시작
3. **유연한 네트워킹**: Unix 소켓, TCP, VsockListener 지원
4. **보안 중심**: TDX Quote 기반 인증서, 키 유도 시스템
5. **시스템 통합**: Systemd watchdog, 로깅, 설정 관리

초기화 순서:

1. **로깅 시스템 → 설정 로딩 → AppState 생성**
2. **키 로딩 → 인증서 클라이언트 생성 → 데모 인증서 생성**
3. **서비스 설정 추출 → 네트워크 바인딩 → 서비스 시작**

4. **Watchdog 시작 → 헬스 체크 루프**

이 구조는 TEE 환경에서의 보안성과 안정성을 보장하면서도 다양한 클라이언트의 요구사항을 충족할 수 있도록 설계되었습니다.